

The idea for this project originated while I was reading Giselle Beiguelman's *Nomadic Poetry* and playing an automated factory game. When I played the game, I also came across a passage that read, "interact with wireless remote networks through cell phones, PDAs, and electronic panels. The fragmentation of distributed interfaces and the mix of words and symbols in alphanumeric writing code create a nonphonetic 'alphabet' that operates through sharing and sampling, situates itself in public spaces, and participates in the instability of contemporary life." (Nomadic Poetry, Pg 286). In the Chinese translation I read, the keywords "cell phones" and "electronic panels" immediately brought to mind the game, while the latter part emphasized the keyword "fragmentation." It's worth noting that "automated factory game" processes basic resources through different machines to create a final product. This reminded me of letters, the most basic and "fragmented" form of expression in English and many other languages, which are combined and varied to create the phrases, poems, novels, and so on that we ultimately see. So initially, I wanted to create something similar, an automated factory game based on letters. However, when I was creating a draft of the page or the ideal experience, I realized the final product was closer to a node-based program, like TouchDesigner or ComfyUI. But I think the core is still about deriving complex products from basic elements through many processes, so there wasn't much change.

I believe that to create good building blocks, you first need to know what the assembled blocks look like, and then work backwards from each step. So I chose a piece I wrote a few years ago that couldn't quite be called a poem and tried to reverse-engineer it. Because I wanted to impose some user restrictions, the letters a-z could only be used once, but many letters in the poem were repeated, so I first added a unit that duplicated the letters. Later, they would definitely be combined into words, then sentences, and so on, but whether it was a word, a sentence, or a

stanza, I wanted this related unit to have no more than 5 input slots; otherwise it would look too complicated. So I added different "synthesizers," which gave these units some scalability. Then, there was no function to delete connecting lines; I realized this was a problem because I accidentally connected two points together incorrectly. At that time, many lines are intertwined, making it difficult to click on them. Therefore, I think clicking on a slot and then deleting the corresponding link is a good idea. Originally, clicking a slot would then cancel the connection if it touched a blank area, which aligns with the behavior of some node-based programs. However, in my usage, I pre-list all the letters I need, which makes it impossible to connect words with letters that I put too far. Therefore, I changed the program so that it doesn't disconnect while moving, but instead uses the right mouse button to disconnect.

After completing the first "example," I felt the entire program was too mechanical and a bit tedious. So, in the second template, I wrote "never gonna give you up," which was interesting enough and didn't take long. This gave me another idea: to incorporate Rick Astley's original "Never Gonna Give You Up" into the program, and when the output contained the phrase "never gonna give you up," pulling down the lever would play the music.

The following modifications were based on ideas and corrections from testing and discussions with Professor Alex Calderwood on Monday (August 24, 2026). First, during testing, a very obvious bug was that I defined all four ports of the letter replicator (previous named Letter Differential) as either input or output, allowing simultaneous input, which was confusing. Also, Alex mentioned wanting to reuse some words, so I changed the replicator to allow all input types to copy themselves into three additional copies to reuse some words. Then there's the random selector; the initial idea was to have the user input two words and write the probability of change. However, in actually building the logic, I found that, firstly, why should users insist on using a

probability other than 50%? Why not just use that one word if they're biased towards either side?

Secondly, all randomness is pseudo-random. For example, if I set the probability to 0.98%, there's still a chance of repeatedly changing the word, and so frequently it doesn't seem like 0.98%. In that case, why not just let the user choose two words to input and then randomly output one? Then there are some restrictions on the user, such as requiring rhyme, consistent first letters or words of all sentences, and spelling that conforms to some dictionaries, etc. But the Blind Assembly Rule and its + version already exist, which is the user cannot see the actual output of a single synthesizer. In discussions with Professor Alex Calderwood, I believe both have their advantages—one more convenient, the other more fun. So I displayed the synthesizer output and made the original version into a limiter (actually two). One displays the number of letters in the current synthesizer, and the other completely hides it. This gives the user the choice between convenience and fun.